Zewail City of Science and Technology

Course Project (20% of Final Grade)
Team Size: 3-4 Students
Stack: (Next.js, Express, MongoDB, MinIO, Redis, Nginx)
ClipSphere– Short Video Social Platform

Objective: ClipSphere is a high-performance, short-video social platform designed as a comprehensive full-stack ecosystem within a local development environment. The architecture centers on a robust Express and MongoDB backend, integrated with a responsive Next.js frontend styled via Tailwind CSS for a seamless user experience. The infrastructure leverages MinIO for S3-compatible object storage and Nginx as a reverse proxy to handle request routing and security across the services. Technical capabilities include real-time event streaming with Socket.io, integrated financial transactions using Stripe, and background job processing via Redis and Bull. The entire system is orchestrated through Docker, ensuring a cohesive and modular application structure from initial API design to final environment configuration.

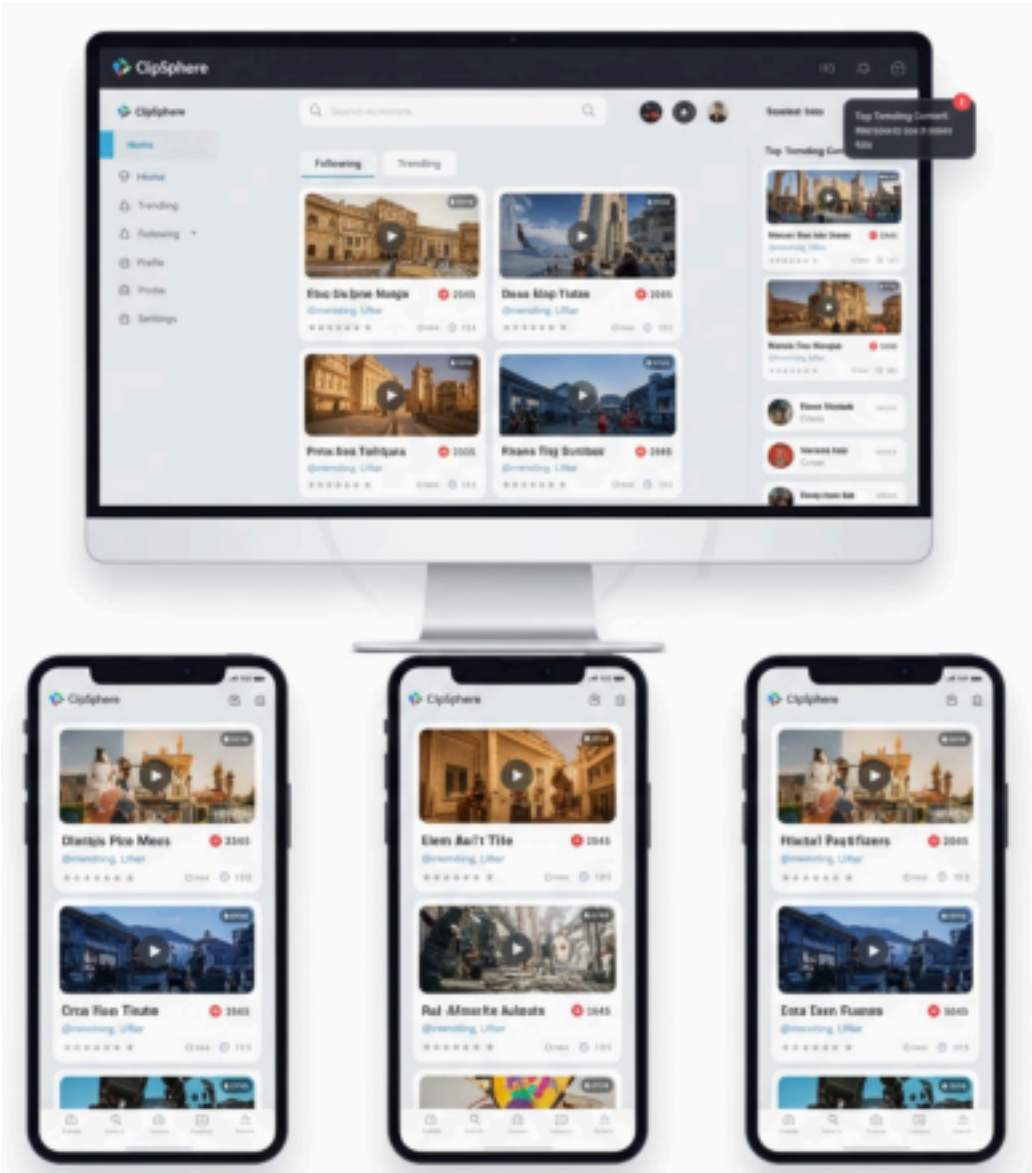
Project Phases Timeline

Project Phases	Deliverables	Grade	Deadl in e
Team Formation	Team size 3-4	**One grade penalty if not submitted on time**	Thursday 26/2/2026
Phase 1	Backend Foundations & Security	**6**	Wednesd ay 18/3/202 6
Phase 2	Next.js Integration & Local Media Pipeline	**4.5**	Sunday 25/3/2026

Phase 3 Real-Time System & Monetization **4.5** Sunday 12/4/2026

Phase 4 Scalability, Nginx & Local DevOps **5** Sunday 10/5/2026

SWAPD352 Web Development Spring-2026



This project is an opportunity to build a professional piece for your portfolio, something you can truly be proud to feature on your resume. Something to add to your GitHub repo when applying for jobs or internships. Your creativity is highly encouraged; feel free to put your own unique spin on the project's UI and Backend features within the bounds of the requirements.

Phase 1 – Backend Foundations & Security (Details Added)

Build the backend infrastructure and core user system, focusing on data integrity and security protocols. Students will use Postman Client to verify all logic locally before any frontend exists.

Required Features:

1. Foundational System Structure

● **Clean Architecture:** Implementation of a "Three-Layer Architecture" separating the entry point (Routes), the orchestration (Controllers), and the business logic (Services). ● **Local Environment Control:** Secure management of local variables using .env, including PORT, MONGODB_URI, and JWT_SECRET.
● **Global Error Handling:** A centralized middleware to catch all async errors, ensuring the server returns consistent JSON error responses instead of crashing or leaking stack traces. ● **Request Logging:** Integration of Morgan or a custom Winston middleware to log every incoming local request (Method, URL, Status) for easier debugging.
● **Security middleware:** integration of express-mongo-sanitize to prevent NoSQL injection attacks during authentication and data updates.

For this section, below are the expected endpoints ensure the server and middleware are functioning correctly:

Method	Endpoint	Description	Middleware/Logic
GET	/health	Basic heartbeat to check if server is running.	Public
GET	/api/v1/admin/health	**(From Sec 7)** Detailed system health (Uptime, DB connection).	protect + restrictTo(admin)

2. Secure Identity Management

● **User Registration: Endpoint to create new accounts with unique email validation.** ● **Input validation:** implementation of schema-based validation using Zod library for all incoming post/patch requests to ensure data integrity before it reaches the database. ● **Password Security:** One-way hashing using Bcrypt with a salt factor of 10 to ensure user credentials are never stored in plain text.

- **Stateless Authentication:** Implementation of JWT (JSON Web Tokens) with a specific expiration time (e.g., 24h). The token should be returned upon successful login and verified via a "protect" middleware (Authentication: "Who are you?") for private routes.
- **Profile Management:** Endpoints to retrieve current user data and update fields. This includes a metadata-only endpoint for avatars to store the MinIO object name before the storage system is fully active.

3. Permission Levels & Authorization (RBAC)

- **Role Definition:** Implementation of a "role" field in the User schema with values: user and admin.
- **Authorization Middleware:** Custom middleware to restrict endpoints based on roles (e.g., only admins can delete other users' videos; only owners can edit their own profile). ● **Authorization (RBAC):** Middleware to restrict specific endpoints (like "Delete any video" or "View all users") based on a user role field (Standard vs. Admin) in the MongoDB document. ● **Profile Management:** Endpoints to retrieve current user data (GET /me) and update fields (PATCH /updateMe), including metadata-only placeholders for future MinIO avatar links.

Permission Levels & Data Ownership

A. The User Role (Standard Ownership)

- **CRUD Operations:** As a standard user, you have full CRUD (Create, Read, Update, Delete) permissions over your **own** data only.
- **Ownership Verification:** Every "Update" or "Delete" request must pass through an **Ownership Middleware**. This logic compares the req.user.id (from the JWT) with the video.ownerId (from the database). If they don't match, the request is rejected with a 403 Forbidden status.

B. The Admin Role (Platform Oversight)

- **Global Delete Permissions:** Admins are granted "Superuser" status. They can bypass the ownership check specifically for the **Delete** action. This allows them to remove community-violating content without being the video's owner.
- **Restricted Edit Permissions:** Even as an Admin, they should generally **not** be able to "Update" your data (like changing your password or bio), ensuring user privacy is maintained.

For Sections 2 and 3, Secure Identity & RBAC, below are expected endpoints:

Method	Endpoint	Description	Middleware/Logic

POST	/api/v1/auth/register	Creates a new user account.	**Zod Validation**, **Bcrypt Hashing**
POST	/api/v1/auth/login	Exchanges credentials for a JWT.	**Bcrypt Compare**

GET	/api/v1/users/me	Retrieves the logged-in user's data.	protect (Authentication)
PATCH	/api/v1/users/updateMe	Updates username, bio, or avatar key.	protect + **Zod Validation**
GET	/api/v1/users/:id	View a specific user's public profile.	Public

4. Database Collections & Schema Design

- Odm integration: mandatory use of the mongoose library to define schemas, enforce validation, and manage database connectivity.
- **User Collection:** Fields for username, email, hashed password, role, bio, and avatar key. Enforce unique indexes on email and username fields at the database level. Add an active boolean field (default: true) to support admin "soft deletes" and an accountstatus field for moderation.
- **Video Collection:** Fields for title, description, owner (ObjectId reference linked to 'user' model), videoURL (key), duration, and timestamps. Add a viewscount field (default: 0) and a status field (e.g., 'public', 'private', 'flagged') for admin oversight.
- **Review Collection:** Fields for rating (1–5), comment text, user (ObjectId), and video (ObjectId). Implement a logic constraint or compound index to prevent a user from submitting multiple reviews on the same video. Use mongoose min and max validators on the rating field to strictly enforce the 1–5 star range.
- **Followers Collection:** Tracking relationships between users (follower vs. following). Add a compound unique index on followerid and followingid to prevent duplicate relationships. Implement pre-save hooks to prevent the "logic bug" where a user attempts to follow themselves.

5. Social Graph & Preferences

- **Follow Logic:** A dedicated schema or sub-document structure in MongoDB to manage relationships. This must prevent a user from following themselves and ensure that unfollowing correctly removes the reference from the database.

- **Granular Notification Schema:** A nested object within the User model dedicated to preferences. It must include boolean toggles for:
  - In-app alerts: (Followers, Comments, Likes, Tips)
  - Email alerts: (Followers, Comments, Likes, Tips)
- **Event Tracking Logic:** Backend logic that checks these preferences during a social action to determine if a notification record should be created or an email should be queued later.

For the Social Graph & Preferences, below are expected endpoints:

Method	Endpoint	Description	Middleware/Logic
POST	/api/v1/users/:id/follow	Follow a user.	protect + **Self-follow check hook**
DELETE	/api/v1/users/:id/unfollow	Unfollow a user.	protect
GET	/api/v1/users/:id/followers	List all users following this account.	Public
GET	/api/v1/users/:id/following	List all accounts this user follows.	Public
PATCH	/api/v1/users/preferences	Update In-app/Email alert toggles.	protect (Auth)

6. Media & Review Logic

- Video Metadata Schema: Initial design for the Video collection, including title, description, duration, and owner ID.
- Duration Constraints: Logic to store and validate that the video length does not exceed 300 seconds (5 minutes).

- Review System Foundation: Schema for video reviews (1-5 star ratings and text) linked to both the User and the Video.

For the Media & Review Logic, below are expected endpoints:

Method	Endpoint	Description	Middleware/Logic
POST	/api/v1/videos	Create metadata (Placeholder for upload).	protect + **Duration < 300s logic**
GET	/api/v1/videos	Feed: List all public videos.	Public
PATCH	/api/v1/videos/:id	Update video title or description.	protect + **Ownership Middleware**
DELETE	/api/v1/videos/:id	Remove video (Owner or Admin).	protect + **Ownership/Admin Logic**
POST	/api/v1/videos/:id/reviews	Submit a 1-5 star review.	protect + **Unique review index**

7: Admin Oversight & Analytics

- **Platform Statistics Endpoint:** A dedicated GET /api/v1/admin/stats route (protected by Admin-only middleware) that uses MongoDB Aggregation Pipelines to return: ○ Total number of users and videos.
    - ○ Total tips processed (related to Phase3).
    - ○ The "Most Active" users of the week.

- **User Management (The "Ban Hammer"):** An endpoint to "Soft Delete" or deactivate a user account (PATCH /api/v1/admin/users/:id/status). This updates the active: false field we added to the schema.
- **Content Moderation Queue:** A route to fetch all videos flagged with low review scores or specifically reported by users.

- **System Health Check:** A simple GET /api/v1/admin/health that returns server uptime, memory usage, and database connection status.

For the Admin Oversight Analytics section, below are expected outlines:

Method	Endpoint	Description	Middleware/Logic
GET	/api/v1/admin/stats	Site analytics (Total users/videos/tips).	protect + restrictTo(admin)
PATCH	/api/v1/admin/users/:id/status	Deactivate/Ban user (Soft delete).	protect + restrictTo(admin)
GET	/api/v1/admin/moderation	Queue of flagged/low-rated content.	protect + restrictTo(admin)

8. Interactive API Documentation (Swagger)

- **OpenAPI Specification:** Integration of swagger-jsdoc and swagger-ui-express to automatically generate documentation from code comments.
- **Endpoint Visualization:** All routes (Auth, Users, Videos, Admin) must be documented with expected request bodies, query parameters, and potential error responses (400, 401, 403, 404).
- **Security Definition:** Configuration of Swagger to support **JWT Bearer** Authentication, allowing the "Try it out" feature to work for protected routes.
- **Access Point:** The documentation must be served at a dedicated local route (e.g., http://localhost:5000/api-docs).

For swagger, expected endpoint below:

Method	Endpoint	Description	Access
GET	/api-docs	Interactive Swagger UI dashboard.	Public

Deliverables

- Everything mentioned above
- GitHub repository with organized folders for routes, controllers, services, models, and middleware.
- Organized folder “models/” directory containing Mongoose Schemas for all collections. ● **Interactive documentation:** integration of swagger-jsdoc and swagger-ui-express. the api must be accessible via /api-docs, showing all schemas, parameters, and authentication headers required for each endpoint.
- **Functional API:** A fully working backend that passes local integration tests for Registration, Login, and RBAC.
- **API Documentation:** A Postman Collection including Environment Variables for base URLs, Scripts for automated token management, and Basic Tests for every endpoint. ● **Database ER Diagram:** A visual map of the MongoDB collections highlighting the relationships between Users, Videos, and Reviews.